МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторные работы
по курсу «Информационный поиск»

Выполнил: Галкин Алексей Дмитриевич
Группа: М8О-408Б-22
Преподаватель: Кухтичев Антон Алексеевич

Москва, 2026

Содержание

1 Введение **3**

2 Лабораторная работа №1: Добыча корпуса документов **3**
2.1 Цель работы . . . 3
2.2 Описание источников . . . 3
2.3 Статистика корпуса . . . 3

3 Лабораторная работа №2: Поисковый робот **3**
3.1 Цель работы . . . 3
3.2 Архитектура и стек технологий . . . 3
3.3 Алгоритм работы . . . 4
3.4 Формат хранения данных . . . 5

4 Лабораторная работа №3: Токенизация **5**
4.1 Цель работы . . . 5
4.2 Основные компоненты . . . 5
4.3 Особенности реализации . . . 6

5 Лабораторная работа №4: Стемминг **6**
5.1 Цель работы . . . 6
5.2 Метод решения . . . 6

6 Лабораторная работа №5: Закон Ципфа **7**
6.1 Цель работы . . . 7
6.2 Теоретические основы . . . 7
6.3 Реализация анализатора . . . 8
6.4 Результаты . . . 8
6.5 Причины расхождений . . . 9
6.6 Выводы . . . 9

7 Лабораторная работа №6: Построение булева индекса для коллекции документов **9**
7.1 Цель работы . . . 9
7.2 Структура документа после обработки . . . 10
7.3 Архитектура индексатора . . . 10
7.3.1 Собственные структуры данных . . . 10
7.3.2 Процесс индексации . . . 10
7.3.3 Бинарный формат индекса . . . 11
7.4 Результаты индексации . . . 11
7.5 Анализ масштабируемости . . . 11

8 Лабораторная работа №7: Реализация булева поиска **12**
8.1 Цель работы . . . 12
8.2 Синтаксис запросов . . . 12
8.3 Архитектура поискового движка . . . 13
8.3.1 Компоненты . . . 13
8.4 Реализация парсера рекурсивного спуска . . . 13
8.5 Примеры выполнения запросов . . . 14
8.6 Интерфейсы поиска . . . 14
8.6.1 Командная строка (CLI) . . . 14

8.6.2 Веб-интерфейс 15

9 Вывод **16**

10 Список литературы **16**

1 Введение

В рамках курса «Информационный поиск» была разработана поисковая система, включающая подсистемы сбора данных (краулер), обработки текста (токенизация, стемминг), индексации и поиска. Сбор данных и веб-интерфейс был реализован на языке Golang, а ядро системы (индексация и поиск) реализовано на языке C++ с использованием собственных структур данных (вектор, хеш-таблица) в соответствии с требованиями курса.

2 Лабораторная работа №1: Добыча корпуса документов

2.1 Цель работы

Собрать и проанализировать корпус текстовых документов объемом не менее 30 000 статей для последующего использования в поисковой системе.

2.2 Описание источников

В качестве предметной области была выбрана тематика живых существ, а именно животные, растения и грибы. Источниками данных послужили:

- русскоязычный раздел Википедии. Выли выбраны 3 категории: Животные по алфавиту, Растения по алфавиту, Грибы по алфавиту. Собрано 65,000 статей.
- журнал о животных (https://animaljournal.ru/) Собрано 282 статей.

2.3 Статистика корпуса

Параметр	**Значение**
Количество документов (всего)	65,282
из Википедии	65,000
из animaljournal	282
Формат хранения	JSONL
Язык	русский (с включениями из других языков)

Таблица 1: Статистика собранного корпуса

3 Лабораторная работа №2: Поисковый робот

3.1 Цель работы

Разработать поисковый робот для автоматического сбора документов, проверки свежести и наличием задержки.

3.2 Архитектура и стек технологий

Робот реализован на языке **Golang** с использованием MongoDB. Основные библиотеки: net/http, golang.org/x/net/html, crypto/sha256. Для обеспечения надёжности работы в HTTPClient реализован механизм повторных попыток с экспоненциальной за-

держкой (retry logic) и ограничение частоты запросов (rate limiting) с использованием мьютексов для потокобезопасности.

Архитектура построена по принципу разделения ответственности и включает три основных уровня: уровень представления (конфигурация и точка входа), уровень управления (центральный менеджер Crawler) и уровень источников данных (клиенты для различных сайтов).

3.3 Алгоритм работы

- **Инициализация**: чтение конфигурации из YAML файла, подключение к MongoDB, создание уникальных индексов для предотвращения дубликатов.
- **Управление источниками**: Проверка наличия активных источников в конфигурации и запуск процесса обхода.
- **Обход Википедии**:
  - Для каждой категории:
    - Формирование API-запроса к MediaWiki API.
    - Получение списка страниц в категории (по 500 записей за запрос).
    - Извлечение page_id и названий страниц.
  - Обработка пагинации через параметр cmcontinue.
  - Для каждой страницы проверка наличия в базе данных по page_id, загрузка HTML.
  - Сохранение в MongoDB.
- **Обход AnimalJournal**:
  - Загрузка первой страницы категории.
  - Извлечение ссылок на статьи по CSS-классу post_title и поиск ссылки на следующую страницу в блоке пагинации.
  - Добавка каждой страницы пагинации в очередь на обработку. Предотвращение повторного посещения через seenPages.
  - Генерация числового ID через хеширование URL (FNV-32a) для каждой ссылки на статью.
  - Загрузка каждой HTML страницы статьи.
  - Сохранение в MongoDB
- **Контроль дубликатов**:
  - Перед загрузкой: Для Википедии проверка page_id, для AnimalJournal URL-хеша.
  - Перед сохранением: Вычисление SHA256 от HTML-содержимого и проверка существования документа с таким же хешем.

3.4 Формат хранения данных

Документы хранятся в MongoDB в коллекции documents со следующей структурой:

- page_id (int) — уникальный идентификатор страницы.
- title (string) — заголовок документа.
- url (string) — канонический URL (очищенный от UTM-меток).
- html (string) — полное HTML-содержимое страницы.
- sha256 (string) — контрольная сумма содержимого.
- category (string) — категория или источник.
- downloaded_at (datetime) — время загрузки.

Для экспорта данных используется формат JSON Lines (JSONL), где каждый документ представлен отдельной строкой в UTF-8-кодировке. Это обеспечивает возможность потоковой обработки файлов любого размера без загрузки в память целиком. При перезапуске робот проверяет наличие уже загруженных документов в базе данных и пропускает их, что обеспечивает идемпотентность операций и позволяет докачивать только новые страницы.

4 Лабораторная работа №3: Токенизация

4.1 Цель работы

Реализовать разбиение текста на токены (слова), которые будут использоваться при индексации Нужно будет учитывать особенности русского языка и многобайтовой кодировки UTF-8.

4.2 Основные компоненты

Токенизатор реализован в виде класса Tokenizer на C++. Основная функция - tokenize, которая принимает строку и возвращает вектор std::string токенов.

Алгоритм работы:

- Обработка UTF-8 - Для каждого байта определяется, является ли он началом символа, и вычисляется полная длина символа в байтах. Это позволяет избежать "разрыва"многобайтовых символов при обработке.
- Распознавание буквенных символов.
- Приведение к нижнему регистру.
- Алгоритм выделения токенов
  - Текст сканируется посимвольно (с учётом реальной длины символов в байтах).
  - При обнаружении буквы она добавляется в буфер текущего слова.
  - При обнаружении не-буквы (пробел, знак препинания и т.д.) текущее слово сохраняется в массив токенов, а буфер очищается.
  - Короткие слова (менее двух символов) отбрасываются как незначимые.

4.3 Особенности реализации

- Эффективность
  - Алгоритм однопроходный - текст сканируется один раз;
  - Не создаются промежуточные копии текста;
  - Используется минимальное количество выделений памяти;
- Устойчивость к ошибкам
  - При обнаружении некорректной UTF-8 последовательности алгоритм продолжает работу;
  - Нераспознанные символы обрабатываются как разделители;
- Расширяемость
  - Легко добавить поддержку других алфавитов (например, латиницы с диакритикой)

5 Лабораторная работа №4: Стемминг

5.1 Цель работы

Реализовать морфологическую нормализацию (стемминг) для улучшения полноты поиска.

5.2 Метод решения

Реализован **алгоритм Портера** для русского языка на C++. Алгоритм последовательно применяет ряд правил для отсечения суффиксов и окончаний.

Перед применением правил в слове выделяются специальные области. Это необходимо, чтобы отсекать только суффиксы, не затрагивая корень.

Области:

1. RV - область после первой гласной буквы в слове
2. R1 - область после первой гласной, которая следует за согласной
3. R2 - область внутри R1, начинающаяся после следующей гласной

Таблица суффиксов:

1. Суффиксы причастий и деепричастий: "вшись "вши "в"
2. Суффиксы прилагательных: "ими "ыми "его "ого"и др.
3. Суффиксы причастий: "ем "нн "вш "ющ"
4. Возвратные суффиксы: "ся "сь"
5. Простые глагольные суффиксы: "ете "нно "йте "ла "ло"и др.
6. Сложные глагольные суффиксы: "ейте "уйте "ыли "ыть "ит"и др.
7. Суффиксы существительных: "иями "ями "ами "ев "ов "ие "ье"и др.

8. Суффиксы превосходной степени: "ейше "ейш"

Последовательность применений правил:

1. Обработка причастий и деепричастий (сначала выполняются более специфические правила)
2. Удаление возвратных суффиксов
3. Обработка прилагательных (пробуются суффиксы прилагательных, потом причастий)
4. Обработка глаголов
5. Обработка существительных
6. Удаление конечного "и"
7. Обработка суффиксов "ость"/"ост"(только в области R2)
8. Обработка суффиксов превосходной степени
9. Нормализация двойного "н"
10. Удаление мягкого знака в области RV

6 Лабораторная работа №5: Закон Ципфа

6.1 Цель работы

Проверить выполнение закона Ципфа на собранном корпусе документов.

6.2 Теоретические основы

Закон Ципфа утверждает, что частота слова в корпусе текстов обратно пропорциональна его рангу:

$$f(r) = \frac{C}{r}$$

где $f(r)$ — частота термина с рангом r, C — константа, зависящая от корпуса.

В логарифмической шкале закон Ципфа представляет собой прямую линию с наклоном -1:

$$\log f(r) = \log C - \log r$$

Более точное приближение даёт обобщённый закон Ципфа–Мандельброта:

$$f(r) = \frac{C}{(r+b)^a}$$

где a и b — параметры, подбираемые по реальным данным.

6.3 Реализация анализатора

Класс `ZipfAnalyzer` реализован на C++ и выполняет следующие функции:

1. Подсчёт частот всех терминов после стемминга с использованием собственной хеш-таблицы `StringMap`
2. Сортировка терминов по убыванию частоты (сортировка вставками)
3. Присвоение рангов
4. Записывает полученные результаты в CSV-файл со столбцами:
   (a) rank
   (b) term
   (c) frequency
   (d) log_rank
   (e) log_frequency

Для построения графика был реализован скрипт на Python

6.4 Результаты

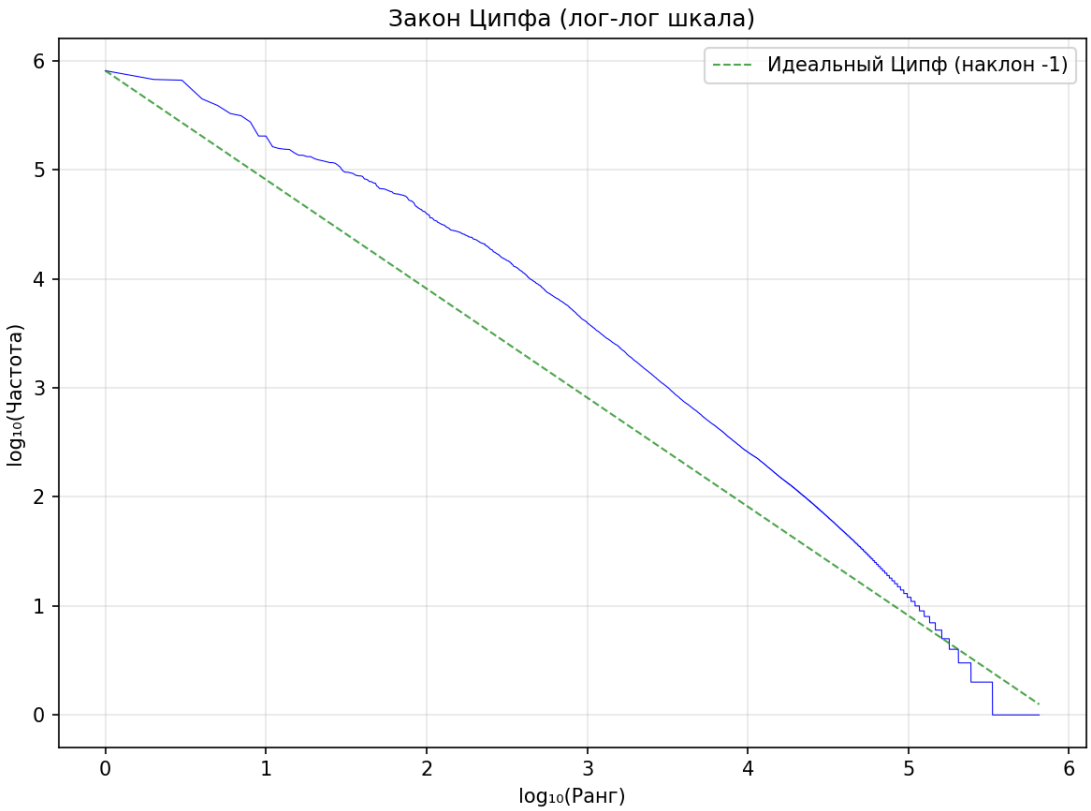

Рис. 1: Распределение частот терминов (закон Ципфа)

Топ-10 самых частотных слов:

Ранг	Терм	Частота
1	в	815668
2	и	677538
3	на	666632
4	с	449801
5	вид	314956
6	род	205073
7	год	157755
8	семейств	154289
9	из	143941
10	без	136810

Таблица 2: Топ-10 частотных терминов

Согласно закону Ципфа: $f \approx \frac{C}{r^s}$, где $s \approx 1$.

Полученный график демонстрирует линейную зависимость с наклоном, близким к -1, что в целом подтверждает выполнение закона Ципфа для собранного корпуса научных текстов.

6.5 Причины расхождений

На графике наблюдаются характерные отклонения от идеального закона Ципфа:

1. Область высоких рангов (частотные слова):

- Слишком много слов с очень высокой частотой. Это "обязательные"термины, которые есть везде.

2. Область низких рангов (редкие слова):

- Наблюдается **ступенчатость** — много слов с частотой 1, 2, 3.
- Частота **выше** предсказанной (хвост толще).
- Причина: огромное количество уникальных терминов

3. Средняя область:

- Хорошее соответствие закону Ципфа.
- Это типично для естественных текстов.

6.6 Выводы

Закон Ципфа выполняется для собранного корпуса с типичными отклонениями, характерными для научных текстов. Отклонения объясняются особенностями предметной области.

7 Лабораторная работа №6: Построение булева индекса для коллекции документов

7.1 Цель работы

Разработать и реализовать инвертированный индекс для поддержки булева поиска в коллекции веб-документов. Индекс должен храниться в бинарном формате с оптимизацией для эффективного выполнения поисковых запросов.

7.2 Структура документа после обработки

Исходный документ (HTML-страница) проходит через конвейер обработки, в результате чего формируется внутреннее представление:

1. **Извлечение текста**: из HTML удаляются все теги, скрипты и стили. Остаётся "чистый"текст.
2. **Токенизация**: текст разбивается на отдельные слова (токены) с учётом UTF-8 кодировки.
3. **Стемминг**: каждый токен приводится к его основе с помощью стеммера Портера для русского языка.

В результате документ представляется как последовательность термов (стеммированных токенов). Для каждого терма сохраняется информация о его позиции(ях) в документе, что позволяет в дальнейшем реализовать поиск по фразам и ранжирование по близости термов.

7.3 Архитектура индексатора

Индексатор реализован на языке C++ без использования STL-контейнеров. Все структуры данных разработаны самостоятельно для обеспечения полного контроля над памятью и форматом хранения.

7.3.1 Собственные структуры данных

- **HashMap**: хеш-таблица с открытой адресацией для хранения словаря термов. Ключом является строка терма, значением — указатель на постинг-лист. При коллизиях используется линейное пробирование. Размер таблицы динамически увеличивается при достижении коэффициента загрузки 0.75.
- **DynArray**: динамический массив для хранения последовательностей чисел (doc_id, позиции термов). Реализует автоматическое расширение с коэффициентом 2 при заполнении. Поддерживает вставку в конец за амортизированное O(1).

7.3.2 Процесс индексации

Индексация выполняется в один проход по коллекции документов:

1. Документы читаются последовательно из JSONL-файла.
2. Для каждого документа:
   - Извлекается текст из HTML (функция stripHTML).
   - Выполняется токенизация с приведением к нижнему регистру.
   - Каждый токен пропускается через стеммер.
   - Для каждого уникального терма в документе формируется постинг, содержащий doc_id, частоту терма и список позиций.
   - Постинг добавляется в хеш-таблицу (если терм новый — создаётся новый постинг-лист).

Важная особенность: поскольку документы обрабатываются в порядке возрастания doc_id (от 0 до N-1), постинг-листы автоматически получаются отсортированными по doc_id. Это позволяет избежать дополнительной сортировки при сохранении индекса.

7.3.3 Бинарный формат индекса

Индекс сохраняется в единый бинарный файл следующей структуры:

```
[0-7]    Magic: "IRIDX003" (8 байт) - сигнатура и версия формата

[8-11]   NumDocs: uint32 - количество документов

[12...]  Документы (NumDocs записей):
         - docId: uint32
         - title_len: uint32
         - title: char[title_len]
         - url_len: uint32
         - url: char[url_len]

[...]    NumTerms: uint32 - количество уникальных термов

[...]    Термы (NumTerms записей):
         - term_len: uint32
         - term: char[term_len]
         - numPostings: uint32
         - Постинги (numPostings записей):
           * docId: uint32
           * count: uint32 (частота терма в документе)
           * snippet_len: uint32
           * snippet: char[snippet_len]
```

Преимущества формата:

- Простота реализации и отладки
- Компактное представление
- Возможность загрузить весь индекс одной операцией чтения

7.4 Результаты индексации

Индексация выполнялась на коллекции из 65,136 документов (смесь русскоязычных статей из Википедии и AnimalJournal.ru).

Параметр	**Значение**
Количество уникальных термов	652,748
Размер файла словаря (index.bin)	2.8 Гб

Таблица 3: Статистические характеристики булева индекса

7.5 Анализ масштабируемости

Текущие ограничения:

- Индекс строится полностью в оперативной памяти, что ограничивает максимальный размер коллекции доступным RAM.

- Однопоточная обработка не использует преимущества многоядерных процессоров.
- При индексации используется наивный подход к выделению памяти.

Прогноз масштабирования:

- **10× данных (650k документов)**: потребуется 2.5-3 ГБ RAM, что всё ещё выполнимо на современных машинах. Время индексации 7-8 минут.
- **100× данных (6.5M документов)**: потребуется 25-30 ГБ RAM, что превышает типичные возможности desktop-систем. Потребуется внешняя сортировка (external merge sort) или распределённая индексация.
- **1000× данных (65M документов)**: необходим распределённый подход (MapReduce, Apache Spark) с индексацией на кластере.

Возможные оптимизации:

1. **Многопоточная токенизация**: разделение документов между потоками с последующим слиянием частичных индексов.
2. **Внешняя сортировка**: при нехватке памяти постинг-листы сортируются на диске.
3. **Блочное сжатие**: применение более эффективных алгоритмов сжатия (LZ4, Zstd) для постинг-листов.
4. **Инкрементальная индексация**: возможность добавления новых документов без перестройки всего индекса.

8 Лабораторная работа №7: Реализация булева поиска

8.1 Цель работы

Разработать поисковый движок, выполняющий булевы запросы (AND, OR, NOT) по построенному индексу, и предоставить интерфейсы для взаимодействия (командная строка и веб-интерфейс).

8.2 Синтаксис запросов

Парсер запросов поддерживает следующий синтаксис (инспирированный поисковыми системами):

- **AND** — логическое И (пересечение множеств).
- **OR** — логическое ИЛИ (объединение множеств).
- **NOT** — логическое НЕ (дополнение множества).
- **Скобки** — изменение приоритета операций.

Примеры запросов:

- `кошка AND собака` — документы, содержащие оба слова

- `кошка OR собака` — документы, содержащие хотя бы одно из слов
- `кошка AND NOTсобака` — документы с "кошка но без "собака"
- `(кошка OR собака) AND NOT мышь` — документы с кошкой или собакой, но без мыши

8.3 Архитектура поискового движка

8.3.1 Компоненты

1. **Загрузчик индекса**: читает бинарные файлы индекса и загружает их в память:
   - Словарь загружается в хеш-таблицу для быстрого доступа по терму
   - Постинг-листы могут загружаться полностью или отображаться через memory-mapped I/O
   - Информация о документах загружается в массив для быстрого доступа по doc_id
2. **Парсер запросов**: реализован методом рекурсивного спуска:
   - `parseExpression()` — обрабатывает операторы OR (самый низкий приоритет)
   - `parseTerm()` — обрабатывает операторы AND
   - `parseFactor()` — обрабатывает операторы NOT и атомы (термы или скобки)
3. **Исполнитель запросов**: выполняет операции над множествами документов:
   - Для каждого терма из индекса извлекается список doc_id
   - Выполняются операции пересечения, объединения и дополнения
   - Результат сортируется для вывода

8.4 Реализация парсера рекурсивного спуска

Парсер обрабатывает запрос в соответствии с формальной грамматикой:

```
<expression> ::= <term> ( 'OR' <term> )*
<term>       ::= <factor> ( 'AND' <factor> )*
<factor>     ::= 'NOT' <factor> | '(' <expression> ')' | <word>
<word>       ::= последовательность букв (латиница или кириллица)
```

Преимущества рекурсивного спуска:

- Прямое отображение грамматики на код
- Легко расширять (добавление новых операторов)
- Понятная обработка приоритетов
- Естественная поддержка скобок

8.5 Примеры выполнения запросов

Для тестирования использовалась коллекция из 65,282 документов.

Запрос	Результатов
кенгуру	228
кенгуру AND австралия	137
кенгуру OR коала	244
кенгуру NOT коала	220
(кенгуру OR вомбат) AND австралия	149
кошка AND собака AND мышь	46
NOT кенгуру	65054

Таблица 4: Примеры выполнения запросов

8.6 Интерфейсы поиска

8.6.1 Командная строка (CLI)

```
# Пакетный режим
$ ./search -index /path/to/index -query "кенгуру && австралия"
Query: кенгуру AND австралия
Results: 137 documents
-1143397447     Динго - собака Австралии, которая одичала. Описание и фото собаки д
-93928591       Большой рыжий кенгуру - гигант Австралии. Описание и фото рыжего ке
8743    Млекопитающие   https://ru.wikipedia.org/wiki/%D0%9C%D0%BB%D0%B5%D0%BA%D0%B
16232   Утконос https://ru.wikipedia.org/wiki/%D0%A3%D1%82%D0%BA%D0%BE%D0%BD%D0%BE%
61824   Сумчатые        https://ru.wikipedia.org/wiki/%D0%A1%D1%83%D0%BC%D1%87%D0%B
80816   Гребнистый крокодил     https://ru.wikipedia.org/wiki/%D0%93%D1%80%D0%B5%D0
84474   Сумчатый волк   https://ru.wikipedia.org/wiki/%D0%A1%D1%83%D0%BC%D1%87%D0%B
84733   Динго   https://ru.wikipedia.org/wiki/%D0%94%D0%B8%D0%BD%D0%B3%D0%BE
86227   Коала   https://ru.wikipedia.org/wiki/%D0%9A%D0%BE%D0%B0%D0%BB%D0%B0
100661  Двурезцовые сумчатые    https://ru.wikipedia.org/wiki/%D0%94%D0%B2%D1%83%D1
147036  Австралийская ехидна    https://ru.wikipedia.org/wiki/%D0%90%D0%B2%D1%81%D1
148095  Бандикуты       https://ru.wikipedia.org/wiki/%D0%91%D0%B0%D0%BD%D0%B4%D0%B
148716  Кенгуровые      https://ru.wikipedia.org/wiki/%D0%9A%D0%B5%D0%BD%D0%B3%D1%8
149229  Мускусная кенгуровая крыса      https://ru.wikipedia.org/wiki/%D0%9C%D1%83%
149443  Кенгуровые крысы (сумчатые)     https://ru.wikipedia.org/wiki/%D0%9A%D0%B5%
149504  Потору  https://ru.wikipedia.org/wiki/%D0%9F%D0%BE%D1%82%D0%BE%D1%80%D1%83
156069  Жесткокрылые    https://ru.wikipedia.org/wiki/%D0%96%D0%B5%D1%81%D1%82%D0%B
256040  Исполинские кенгуру     https://ru.wikipedia.org/wiki/%D0%98%D1%81%D0%BF%D0
587036  Большой желтохохлый какаду      https://ru.wikipedia.org/wiki/%D0%91%D0%BE%
774327  Белая акула     https://ru.wikipedia.org/wiki/%D0%91%D0%B5%D0%BB%D0%B0%D1%8
  ... and 117 more

# Интерактивный режим
$ ./engine search -index index.bin
Index loaded: 652748 terms, 65282 documents
Interactive search mode. Enter queries (one per line), Ctrl+D to exit.
кенгуру
Query: кенгуру
```

```
Results: 228 documents
-1143397447 Динго - собака Австралии, которая одичала. Описание и фото собаки динго
-223429113 Манчкин https://animaljournal.ru/article/koshka_manchkin
-93928591 Большой рыжий кенгуру - гигант Австралии. Описание и фото рыжего кенгуру
8743 Млекопитающие https://ru.wikipedia.org/wiki/%D0%9C%D0%BB%D0%B5%D0%BA%D0%BE%D0%
16232 Утконос https://ru.wikipedia.org/wiki/%D0%A3%D1%82%D0%BA%D0%BE%D0%BD%D0%BE%D1
21929 Тарпан https://ru.wikipedia.org/wiki/%D0%A2%D0%B0%D1%80%D0%BF%D0%B0%D0%BD
61824 Сумчатые https://ru.wikipedia.org/wiki/%D0%A1%D1%83%D0%BC%D1%87%D0%B0%D1%82%D
80816 Гребнистый крокодил https://ru.wikipedia.org/wiki/%D0%93%D1%80%D0%B5%D0%B1%D0
84474 Сумчатый волк https://ru.wikipedia.org/wiki/%D0%A1%D1%83%D0%BC%D1%87%D0%B0%D1
84733 Динго https://ru.wikipedia.org/wiki/%D0%94%D0%B8%D0%BD%D0%B3%D0%BE
86227 Коала https://ru.wikipedia.org/wiki/%D0%9A%D0%BE%D0%B0%D0%BB%D0%B0
93909 Трубкозуб https://ru.wikipedia.org/wiki/%D0%A2%D1%80%D1%83%D0%B1%D0%BA%D0%BE%
95651 Фолклендская лисица https://ru.wikipedia.org/wiki/%D0%A4%D0%BE%D0%BB%D0%BA%D0
100661 Двурезцовые сумчатые https://ru.wikipedia.org/wiki/%D0%94%D0%B2%D1%83%D1%80%
106871 Тираннозавр https://ru.wikipedia.org/wiki/%D0%A2%D0%B8%D1%80%D0%B0%D0%BD%D0%
127566 Панголины https://ru.wikipedia.org/wiki/%D0%9F%D0%B0%D0%BD%D0%B3%D0%BE%D0%BB
130618 Стеллерова корова https://ru.wikipedia.org/wiki/%D0%A1%D1%82%D0%B5%D0%BB%D0%
135921 Обыкновенный шакал https://ru.wikipedia.org/wiki/%D0%9E%D0%B1%D1%8B%D0%BA%D0
147036 Австралийская ехидна https://ru.wikipedia.org/wiki/%D0%90%D0%B2%D1%81%D1%82%
148095 Бандикуты https://ru.wikipedia.org/wiki/%D0%91%D0%B0%D0%BD%D0%B4%D0%B8%D0%BA
  ... and 208 more
```

8.6.2 Веб-интерфейс

Веб-интерфейс реализован на языке Golang с использованием стандартной библиотеки net/http

- **Главная страница** (`/`):
  - Центральное поле ввода поискового запроса с автоматическим фокусом
  - Подсказка по синтаксису запросов (поддержка операторов AND, OR, NOT и скобок)
- **Страница результатов** (`/?q=...`):
  - Форма поиска в верхней части страницы (сохраняется введённый запрос)
  - Счётчик найденных документов (`Найдено: X документов`)
  - Список результатов (до 50 документов на странице), где каждый результат содержит:
    * Заголовок документа в виде ссылки на оригинальную статью, на которую можно нажать
    * Идентификатор документа (Document ID) для отладки
  - Адаптивная вёрстка для комфортного просмотра на мобильных устройствах

9 Вывод

В ходе лабораторных работ была разработана поисковая система с использованием собственных структур данных. Были реализованы и исследованы ключевые компоненты поисковой системы: анализ и сбор документов, индексация и токенизация, стемминг и булев поиск, практическая проверка закона Ципфа. Система успешно справляется с поиском по корпусу из 65282 документов

10 Список литературы

1. Wikipedia API Documentation. — URL: `https://www.mediawiki.org/wiki/API`

2. Маннинг К., Рагхаван П., Шютце Х. Введение в информационный поиск. — М.: Издательский дом «Вильямс», 2011. — 528 с.

3. Porter M. F. An algorithm for suffix stripping // Program: electronic library and information systems. — 1980. — Vol. 14, No. 3. — P. 130–137.

4. Zipf G. K. Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology. — Cambridge, Massachusetts: Addison-Wesley Press, 1949.

5. Кнут Д. Искусство программирования. Том 3. Сортировка и поиск. — 2-е изд. — М.: Издательский дом «Вильямс», 2007. — 832 с.

6. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Издательский дом «Вильямс», 2013. — 1296 с.

7. Baeza-Yates R., Ribeiro-Neto B. Modern Information Retrieval: The Concepts and Technology behind Search. — 2nd ed. — Addison-Wesley Professional, 2011. — 944 p.

8. Сегалович И. В. Как работают поисковые системы // Мир Internet. — 2002. — № 10.

9. Snowball: A language for stemming algorithms. — URL: `https://snowballstem.org/`